Smart Code Tutor

Development of a Multi-Agent System Built on Fine-Tuned CodeT5 for Personalized Code Explanation

---

CS5202 – Generative AI & Large Language Models

TEAM – 13

Name	ID
Kush Jalan	SE23UCSE244
Vanshika Batra	SE23UMCS062

---

1. Problem Statement and Motivation

Artificial Intelligence is increasingly being used in education and programming learning. One important application of large language models is automatic code explanation where a system reads source code and generates natural language explanations.

Though models such as CodeT5 perform well in code understanding and summarization tasks but the generated explanations are usually technical and assume prior programming knowledge. As a result, beginners may find them difficult to understand.

The main problem addressed in this project is the lack of personalized and beginner-friendly code explanations. The existing systems often use a single explanation style regardless of a user's experience level.

This issue mainly affects beginner programmers, students and self-learners who often struggle to understand technical explanations and programming logic.

This problem is important because poor understanding during early learning stages can slow learning progress and reduce confidence. Therefore, our project Smart Code Tutor was developed using a fine-tuned CodeT5 model and a multi-agent framework to generate simpler, clearer and personalized explanations.

2. Method

The Smart Code Tutor system was developed through four major stages such as dataset preparation, CodeT5 fine-tuning, multi-agent refinement and Streamlit deployment.

2.1. Dataset Preparation

The dataset was prepared using a hybrid strategy. Code snippets were collected from CodeSearchNet and few samples were prepared manually and then beginner-friendly explanations were generated using Gemini and ChatGPT-assisted processing.

The dataset included:

- Python
- Java
- JavaScript
- Go

Each dataset sample followed:

Input: Source code

Output: Beginner-friendly explanation

The processing steps included filtering invalid code, explanation generation, cleaning and conversion into JSON format.

2.2. Fine-Tuning CodeT5

The pretrained model selected for training was:

`Salesforce/codet5-small`

CodeT5 was selected because it is designed for code understanding and generation tasks and making it suitable for transforming source code into natural language explanations.

Fine-tuning was performed using Hugging Face Transformers with a sequence-to-sequence learning approach. The prepared dataset contained source code as input and beginner-friendly explanations as target output.

The objective of fine-tuning was to adapt the pretrained model to generate simpler and educational explanations rather than short technical summaries.

The fine-tuned model was saved locally and integrated into the Smart Code Tutor pipeline where it acts as the baseline explanation generator before multi-agent refinement.

2.3. Multi-Agent Framework

A four-agent architecture was implemented using CrewAI.

Agent	Responsibility
Technical Code Analyst	Generates baseline explanation using local fine-tuned CodeT5.
Educational Personalization Expert	Adapts explanations according to user level.
Algorithm Specialist	Provides time and space complexity.
Content Quality Lead	Combines all outputs into a final structured explanation.

Table 1: Four-Agent Architecture

2.4. System Pipeline

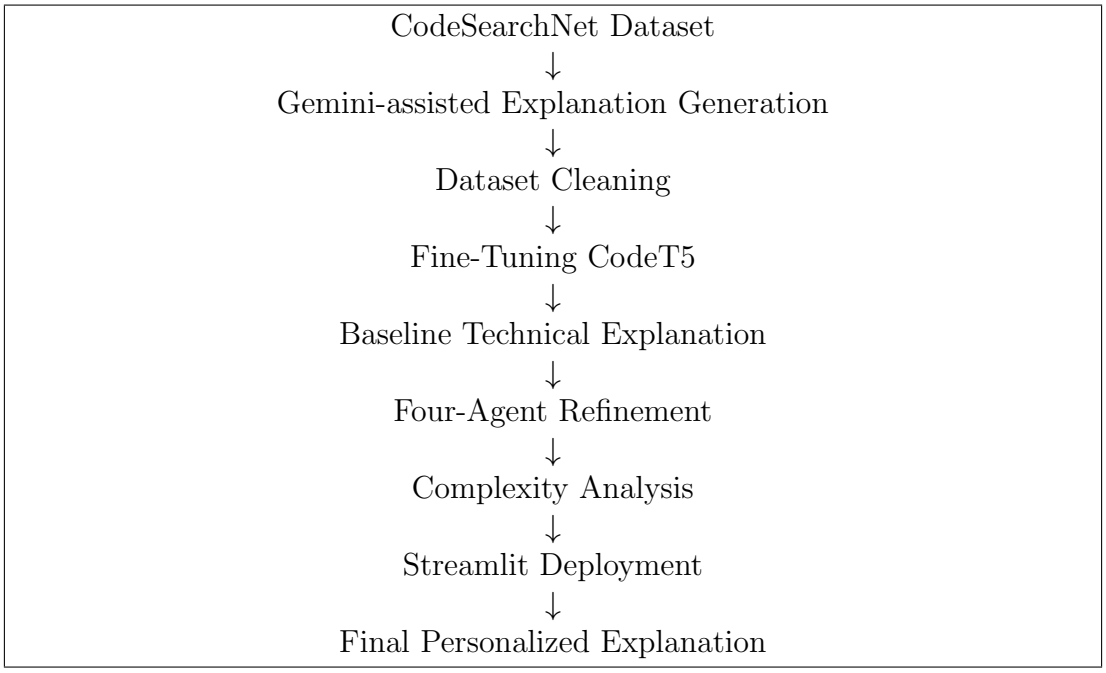

Figure 1: Smart Code Tutor Pipeline

2.5. Deployment

The final Smart Code Tutor system was deployed using Streamlit to provide an interactive and user-friendly interface. Users can paste source code and select a learning level such as Beginner, Intermediate or Advanced.

The application loads the locally saved fine-tuned CodeT5 model and generates an initial explanation. This output is then passed through the multi-agent framework for refinement and complexity analysis.

The final response provides a simplified and personalized explanation, making the system useful for students and beginner programmers.

3. Experiments and Results

Experiments were conducted to evaluate whether the multi-agent architecture improves explanation quality compared to direct CodeT5 output. The evaluation focused on measuring the effectiveness of the fine-tuned model and understanding how the additional refinement agents contributed to explanation quality.

Different dataset sizes were used during training to observe changes in validation loss and ROUGE metrics. The experiments also analyzed whether increasing training data improved the model's ability to generate educational and beginner-friendly explanations.

3.1. Experimental Setup

Base Model	Salesforce/codet5-small
Framework	Hugging Face
Agent System	CrewAI
Deployment	Streamlit
Learning Modes	Beginner, Intermediate, Advanced

Table 2: Experimental Setup

3.2. Fine-Tuning Results

The fine-tuned CodeT5 model was evaluated using validation loss and ROUGE scores. ROUGE metrics were used to compare the generated explanations with the target beginner-friendly explanations.

Samples	Validation Loss	ROUGE-1	ROUGE-2	ROUGE-L
50	5.266997	10.49	2.08	9.46
100	3.668875	13.94	3.96	12.48
500	2.335217	27.42	15.66	24.30
1K	1.870388	33.29	20.58	30.07
5K	1.477594	45.97	33.52	43.41
10K	1.369579	48.48	35.59	45.57

Table 3: CodeT5 fine-tuning performance with increasing dataset size

The final scores after training with all samples were:

- Validation Loss: 1.2258
- ROUGE-1: 51.19
- ROUGE-2: 38.43
- ROUGE-L: 48.23

- ROUGE-Lsum: 48.26
- Epochs: 3

Samples	Training Loss	Validation Loss	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
50	No log	5.266997	10.49	2.08	9.46	9.48
100	No log	3.668875	13.94	3.96	12.48	12.50
500	2.9890	2.335217	27.42	15.66	24.30	24.25
1K	2.4515	1.870388	33.29	20.58	30.07	30.09
5K	1.6575	1.477594	45.97	33.52	43.41	43.43
10K	1.7476	1.369579	48.48	35.59	45.57	45.55

Figure 2: Final ROUGE scores after training with all samples

The results show that validation loss decreased and ROUGE scores improved with larger training datasets, indicating better model learning. The final ROUGE-1 score of 51.19 and ROUGE-L score of 48.23 demonstrate improved explanation quality and more structured outputs. Overall, increasing the training data improved explanation quality and helped produce more structured and educational responses.

3.3. Sample Output

Input:

```
def is_even(num):
    return num % 2 == 0
```

Generated explanation:

This function checks whether a number is divisible by two. If the remainder becomes zero, the function returns True; otherwise False.

Time Complexity: O(1)

Space Complexity: O(1)

3.4. Comparison Study

Feature	CodeT5 Only	CodeT5 + Agents
Explanation Quality	Moderate	Improved
Personalization	Limited	High
Complexity Analysis	Missing	Included
Learning Support	Basic	Better

Table 4: Comparison Results

4. Analysis

The Smart Code Tutor system successfully achieved its objective of generating beginner-friendly and personalized code explanations. The combination of fine-tuned CodeT5 and multi-agent refinement improved explanation quality compared to direct model output.

Experimental results showed that increasing training data reduced validation loss and improved ROUGE scores by indicating better learning performance.

The modular architecture was another advantage because dataset preparation, model training and deployment remained independent by making future improvements easier.

Some limitations were also observed. The quality of generated explanations depends heavily on dataset quality and the deployed system currently focuses mainly on Python code.

Overall, the system addresses the stated problem effectively by producing explanations that are clearer, more structured and easier for beginners to understand.

5. References

1. Wang et al., "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," arXiv:2109.00859, 2021.

2. Husain et al., "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," arXiv:1909.09436, 2019.

3. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.

4. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

5. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.